Exercise 1: Containerizing a C++ Microservice using Docker

Objective

The objective of this exercise is to package a simple C++ microservice into a Docker container. The application is built, containerized, and executed using Docker in an Ubuntu-based cloud lab environment.

System Requirements

- Ubuntu-based Cloud Lab
- Docker installed and running
- No SSH access required
- Web-based terminal access

Project Directory Structure

```
cpp-docker/
 ├── server.cpp
 ├── Dockerfile
```

Step 1: Creating the C++ Microservice

A simple C++ HTTP server is created which listens on port 8080 and responds with a text message.

Screenshot Placeholder:

```
 GNU nano 6.2                                                server.cpp
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in address{};
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(8080);

    bind(server_fd, (struct sockaddr*)&address, sizeof(address));
    listen(server_fd, 3);

    std::cout << "Server running on port 8080..." << std::endl;

    while (true) {
        int client = accept(server_fd, nullptr, nullptr);

        const char* response =
            "HTTP/1.1 200 OK\n"
            "Content-Type: text/plain\n\n"
            "Hello from C++ Docker container!";

        send(client, response, strlen(response), 0);
        close(client);
    }
}
```

Step 2: Creating the Dockerfile

A Dockerfile is written to define the base image, install required dependencies, compile the C++ application, and run it inside a container.

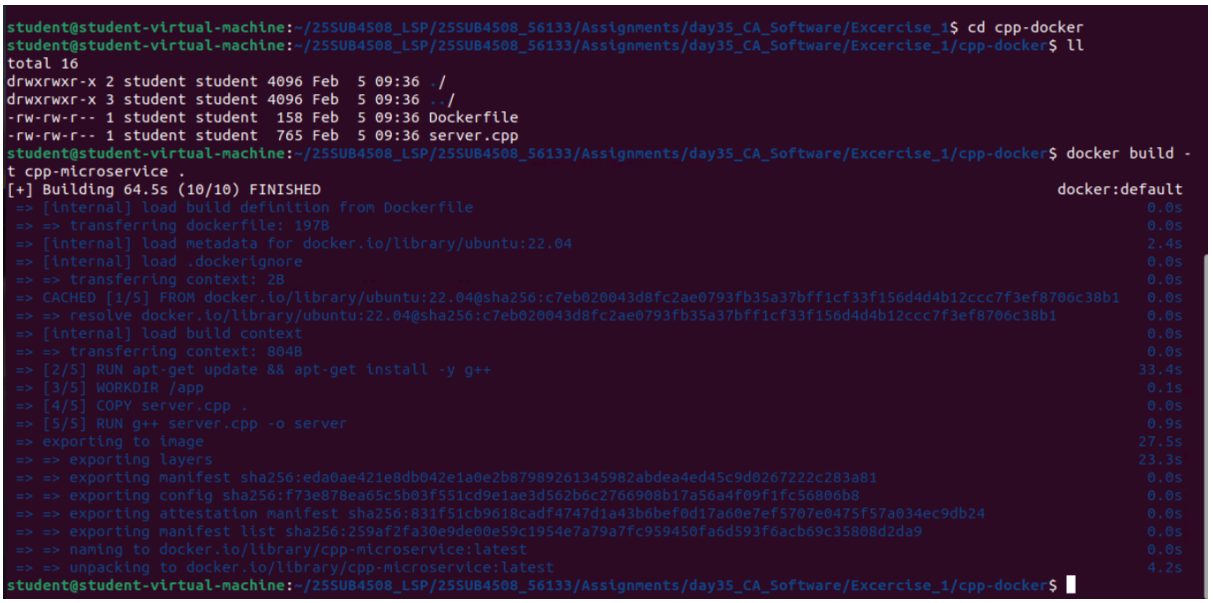
Screenshot Placeholder:

```
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/As...   student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/As...
 GNU nano 6.2                                                Dockerfile
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y g++

WORKDIR /app

COPY server.cpp .

RUN g++ server.cpp -o server

EXPOSE 8080

CMD ["./server"]
```

Step 3: Building the Docker Image

The Docker image is built using the Docker CLI command. This process packages the application and its dependencies into a container image.

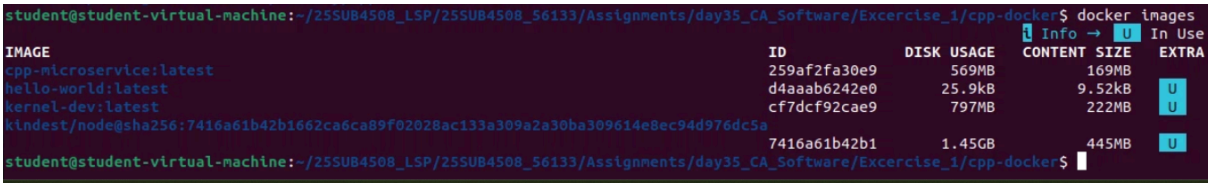
Screenshot Placeholder:



Step 4: Running the Docker Container

The Docker container is run by mapping the container port to the host port. The application starts successfully inside the container.

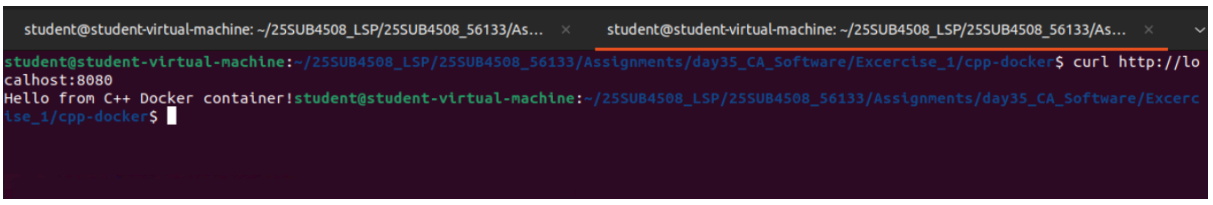
Screenshot Placeholder:



Step 5: Testing the Microservice

The running microservice is tested using a browser or curl command to verify the output.

Screenshot Placeholder:



Conclusion

In this exercise, a C++ microservice was successfully containerized using Docker. The Docker image was built and executed, and the service was tested via an exposed port.